

COS80023 Big Data – Lab 7**Lab 7: Pass Task 7 – Machine Learning****Student Name: Arun Ragavendhar Arunachalam Palaniyappan****Student ID: 104837257****Subtask 7.1****1) What is the difference between supervised and unsupervised learning? When would you use them? Give an example.**

Supervised learning means the data comes with the correct answer (a “label”), so we can train a model to predict that answer for new data. In practice, we split labelled data into a training part to fit the model and a test part to check it on unseen cases. Common supervised tasks are classification (predict a class like “spam/not spam”) and regression (predict a number like price). Unsupervised learning has no labels, so we look for structure such as groups or patterns in the data. A common unsupervised task is clustering (group similar items together). Supervised learning is often preferred when we can afford to label a sample because it tends to be more accurate. Unsupervised learning is useful when labels are too costly or unavailable, or when we want to explore the data first. Example supervised case: using past labelled emails to train a spam classifier. Example unsupervised case: clustering customers into segments based on their behaviour to design targeted offers.

2) What is overfitting and how do you detect it?

Overfitting is when a model learns the training data too closely, including noise, so it performs poorly on new data. A classic sign is high accuracy on training data but lower accuracy on test data. You detect it by holding out a test set the model never sees during training and comparing results. You can also use cross-validation to check stability across multiple random splits. Simple models and regularisation reduce overfitting by limiting how complex the model can get. Feature selection or dimensionality reduction can help too. Visual checks (learning curves) also reveal gaps between training and validation performance.

3) How does cross-validation help with overfitting? Explain the principle

Cross-validation splits the labelled data into k folds (often $k = 10$). We train on $k-1$ folds and test on the remaining fold; then we rotate which fold is the test set and repeat k times. This gives k test results on different held-out data, which is more reliable than a single split. The average (and spread) of those results shows how well the model generalises. Using stratified folds keeps class proportions similar across folds, which is especially helpful for classification. Cross-validation helps spot overfitting because a model that memorises one split will not perform consistently across all folds. We typically select the model (or hyperparameters) that achieves the best average validation performance. Finally, we retrain on all data with those settings before deployment.

4) What different aspects of classification quality do sensitivity, specificity and accuracy measure? Why do we need several measures for classification quality?

In a binary classifier with a confusion matrix, accuracy is the share of all correct predictions out of all cases. Sensitivity (recall/true positive rate) is the share of actual positives the model correctly finds. Specificity (true negative rate) is the share of actual negatives the model correctly rejects. We need several measures because accuracy alone can be misleading when classes are imbalanced (e.g., rare positives). Sensitivity matters when missing a positive is costly (e.g., illness). Specificity matters when false alarms are costly (e.g., fraud flags). Looking at all three gives a balanced view of model quality. (Lecture 7: confusion matrix and formulas for accuracy, sensitivity, specificity).

Subtask 7.2

Exercise 1

Commands to run:

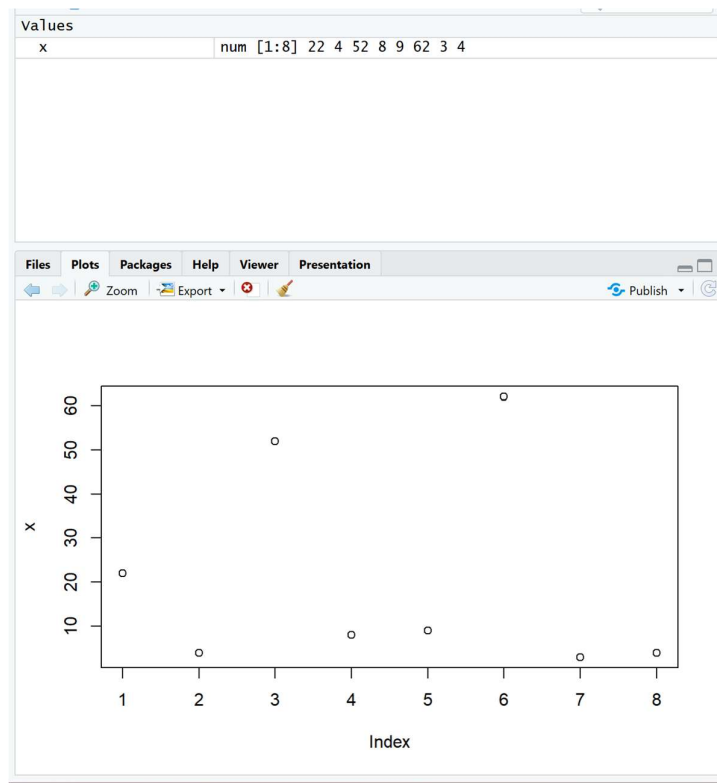
```
x <- c(22, 4, 52, 8, 9, 62, 3, 3, 4)
x
mean(x)
sd(x)
plot(x)
hist(x)
```

Q1. What do these commands do?

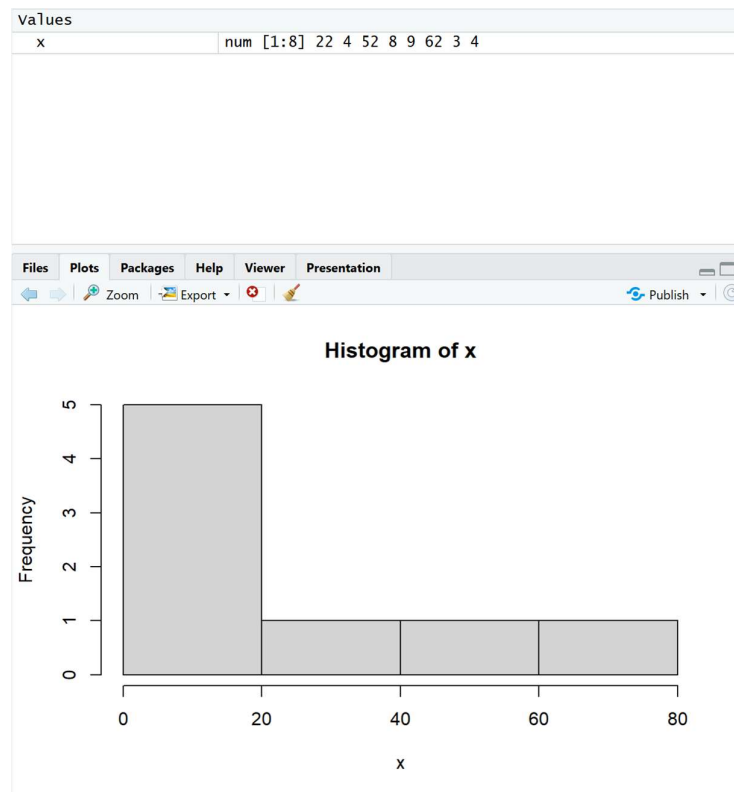
- `x <- c(...)` creates a numeric vector.
- `x` prints the vector.
- `mean(x)` calculates the average of the numbers.
- `sd(x)` calculates how spread out the numbers are from the average.
- `plot(x)` draws a simple scatter/line plot of the values.
- `hist(x)` draws a histogram to show how often values fall into ranges (bins).

Q2. What are standard deviation and histograms?

- **Standard deviation** is a single number that tells you how far, on average, the values are from the mean; higher means more spread.



- A **histogram** is a bar chart that groups values into ranges (bins) and shows how many values fall in each range.



Exercise 2

Task: Think of ten friends. Compute average age and standard deviation, and plot the ages.

Example R (replace the sample ages with your own 10 numbers):

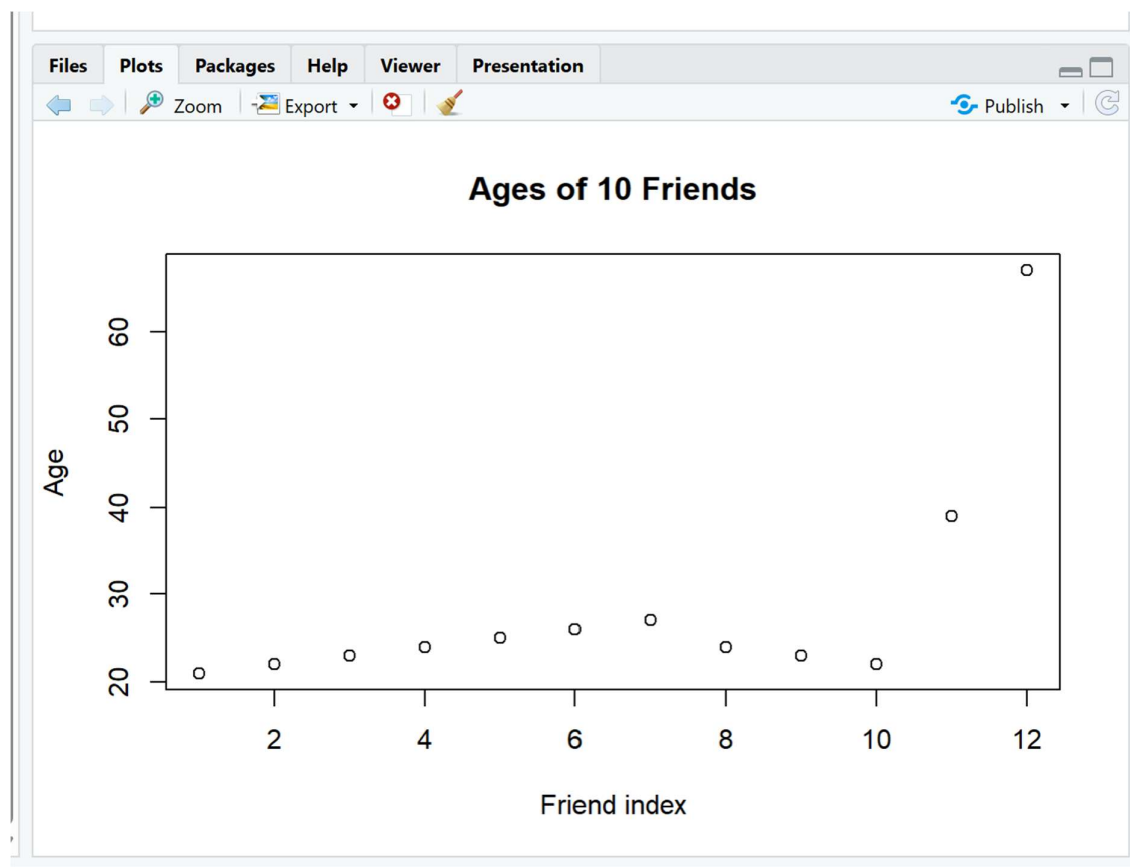
```
ages <- c(21, 22, 23, 24, 25, 26, 27, 24, 23, 22, 39, 67)
```

```
mean(ages)
```

```
sd(ages)
```

```
plot(ages, ylab = "Age", xlab = "Friend index", main = "Ages of 10 Friends")
```

```
> ages <- c(21, 22, 23, 24, 25, 26, 27, 24, 23, 22, 39, 67)
> mean(ages)
[1] 28.58333
> sd(ages)
[1] 12.98571
> plot(ages, ylab = "Age", xlab = "Friend index", main = "Ages of 10 Friends")
>
```



Exercise 3

Load iris data:

```
iris.data <- read.csv(file.choose())
```

```
iris.data
```

Explore columns and values:

```
names(iris.data)
```

```
iris.data$sepal_length
```

```
iris.data$species
```

```
iris.data[1, 1]
```

```
iris.data[1:3, 1]
```

```
iris.data[1:3, ]
```

```
class <- iris.data[1, 5]
```

```
class
```

Q3. Create iris.subset without the species column (annotation).

To create iris.subset without the annotation (species column), I used:

```
iris.subset <- iris.data[, 1:4]
```

```
148      6.5      3.0      5.2      2.0 virginica
149      6.2      3.4      5.4      2.3 virginica
150      5.9      3.0      5.1      1.8 virginica
```

```
> iris.subset <- iris.data[, 1:4]
```

```
> iris.subset
```

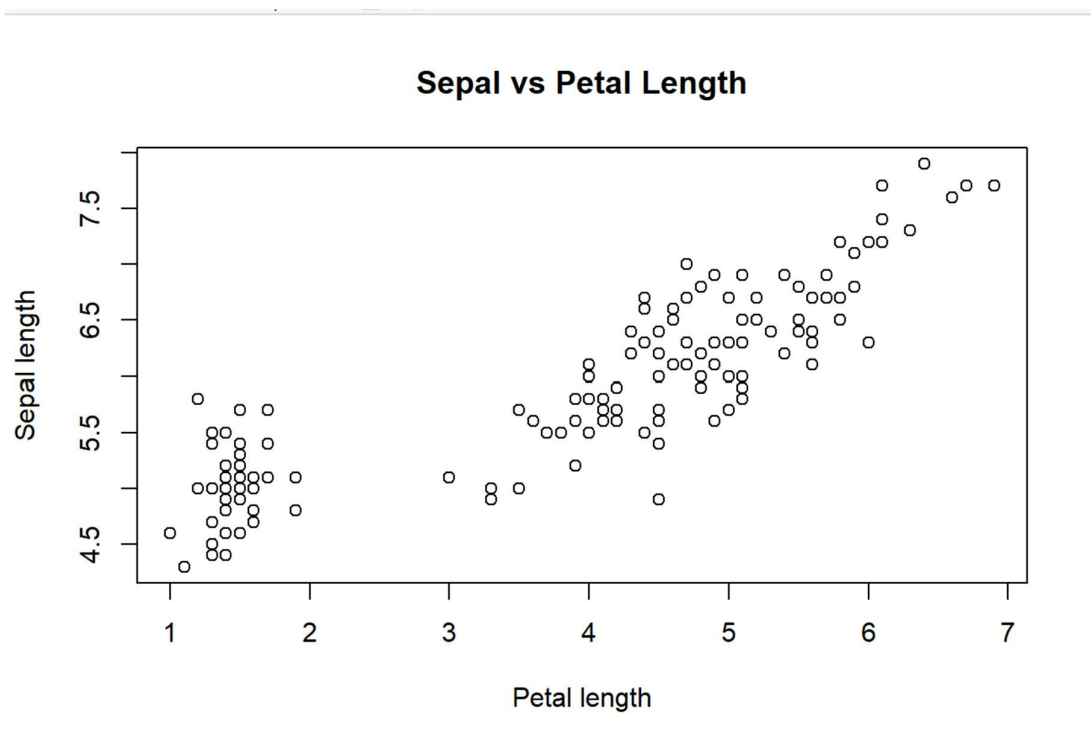
```
  sepal_length sepal_width petal_length petal_width
1          5.1          3.5          1.4          0.2
2          4.9          3.0          1.4          0.2
3          4.7          3.2          1.3          0.2
4          4.6          3.1          1.5          0.2
5          5.0          3.6          1.4          0.2
6          5.4          3.9          1.7          0.4
7          4.6          3.4          1.4          0.3
8          5.0          3.4          1.5          0.2
9          4.4          2.9          1.4          0.2
10         4.9          3.1          1.5          0.1
11         5.4          3.7          1.5          0.2
12         4.8          3.4          1.6          0.2
13         4.8          3.0          1.4          0.1
14         4.3          3.0          1.1          0.1
15         5.8          4.0          1.2          0.2
16         5.7          4.4          1.5          0.4
17         5.4          3.9          1.3          0.4
18         5.1          3.5          1.4          0.3
19         5.7          3.8          1.7          0.3
20         5.1          3.8          1.5          0.3
21         5.4          3.4          1.7          0.2
22         5.1          3.7          1.5          0.4
23         4.6          3.6          1.0          0.2
24         5.1          3.3          1.7          0.5
25         4.8          3.4          1.9          0.2
26         5.0          3.0          1.6          0.2
27         5.0          3.4          1.6          0.4
28         5.2          3.5          1.5          0.2
29         5.2          3.4          1.4          0.2
30         4.7          3.2          1.6          0.2
31         4.8          3.1          1.6          0.2
32         5.4          3.4          1.5          0.4
33         5.2          4.1          1.5          0.1
34         5.5          4.2          1.4          0.2
35         4.9          3.1          1.5          0.1
```

Exercise 4

Scatterplot of two columns:

```
plot (iris.data$sepal_length ~ iris.data$petal_length,
      xlab = "Petal length", ylab = "Sepal length",
      main = "Sepal vs Petal Length")

> plot (iris.data$sepal_length ~ iris.data$petal_length,
+       xlab = "Petal length", ylab = "Sepal length",
+       main = "Sepal vs Petal Length")
>
>
```



Q4. What do you think, is there a correlation between these columns? How strong do you think it is? Is it positive or negative?

From the scatterplot, the points generally rise from left to right, which suggests a positive relationship. The pattern is fairly tight, so the correlation looks moderate to strong rather than weak.

Q5. Try to plot other pairs of columns of the Iris dataset. Which pairs have the strongest (positive or negative) correlation, do you think?

In the classic iris dataset, petal length and petal width usually show the strongest positive relationship. Some pairs (like sepal width with certain others) can look weaker or even

slightly negative. The best way to judge is to draw a few scatterplots and compare how tightly points follow an upward or downward line.

Q6. Compute the correlation matrix

```
cor(iris.subset)
```

```
> cor(iris.subset)
      sepal_length sepal_width petal_length petal_width
sepal_length    1.0000000 -0.1093692  0.8717542  0.8179536
sepal_width     -0.1093692  1.0000000 -0.4205161 -0.3565441
petal_length     0.8717542 -0.4205161  1.0000000  0.9627571
petal_width      0.8179536 -0.3565441  0.9627571  1.0000000
> |
```

My guess was correct and has been verified from the correlation matrix.

Petal length and petal width have the strongest correlation of +0.9627571.

Then, did a scatter plot to confirm the same.

```
plot (iris.data$petal_length ~ iris.data$petal_width,
      xlab = "Petal length", ylab = "Petal width",
      main = "Petal length vs Petal width")
```

